Results

This document summarises the comparison of four deep-learning architectures for multiclass human activity recognition: a fully connected network (DNN), a vanilla recurrent network (RNN), a 1D convolutional network (CNN) and a Long Short-Term Memory network (LSTM). Every model receives the same 561 engineered features per sample and predicts one of six activities: LAYING, SITTING, STANDING, WALKING, WALKING_DOWNSTAIRS and WALKING_UPSTAIRS.

Experimental setup and hyperparameter choices

Data split and preprocessing. The original training set is split into training (5551 samples, 16 subjects) and validation (1801 samples, 5 subjects) with a `GroupShuffleSplit` by subject. Grouping by subject prevents the same person from appearing in both sets, so the validation score estimates how the model generalises to new people, which is the real use case. The features are standardised with a `StandardScaler` fitted only on the training split. The test set (2947 samples) is used only for the final evaluation, never for model selection.

Training configuration. All models share the same configuration so that the comparison is fair:

- **Optimiser: Adam with learning rate 10^{-3}.** Adam adapts the step size per parameter and works well with this default learning rate for very different architectures (dense, convolutional and recurrent), so no per-model tuning is needed for a first comparison.
- **Batch size 64.** With 5551 training samples this gives about 87 updates per epoch: enough to converge in a few dozen epochs, with some gradient noise that acts as regularisation, while still using the GPU efficiently.
- **At most 50 epochs, early stopping with patience 8 and best-weight restoring.** The 50-epoch limit is only an upper bound (no model reached it). Early stopping monitors the validation loss, and a patience of 8 epochs tolerates the noisy validation curves of the CNN and the recurrent models without stopping too early. At the end the weights of the best validation epoch are restored, so model selection is based only on validation data.
- **Gradient clipping at norm 1.0.** The RNN and LSTM are unrolled over 561 steps, where exploding gradients are likely; clipping keeps the updates bounded and costs nothing for the other models.
- **Dropout 0.3 in every model.** A moderate value that regularises without slowing convergence too much.
- **Seed 42** for the main comparison (weights initialisation, data shuffling and dropout).

Architectures. The default sizes were chosen to keep every model small enough to train in seconds while still having a reasonable capacity:

- **DNN:** 561 → 256 → 128 → 6, each hidden layer with BatchNorm, ReLU and dropout. The funnel shape compresses the features progressively; BatchNorm stabilises and speeds up training.
- **RNN and LSTM:** one layer with hidden size 64. Each sample is read as a sequence of 561 steps with one value per step, and the last hidden state goes to a linear classifier. A single layer keeps the sequential cost low, since each extra layer adds another pass over the 561 steps.
- **CNN:** three blocks Conv1d → BatchNorm → ReLU → MaxPool(2) with 32, 64 and 128 filters and kernel size 5, followed by global average pooling. Each pooling halves the length (561 → 280 → 140 → 70), and doubling the filters compensates for it. Global average pooling avoids a large dense layer and keeps the parameter count low.

Training ran on an NVIDIA RTX 3050 Laptop GPU. The metrics below are computed on the test set.

Main comparison

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1	Parameters	Training Time	Inference Time
DNN	0.9525	0.9542	0.9510	0.9520	0.9524	178310	6.63 s	0.0029 s
RNN	0.7642	0.7594	0.7542	0.7557	0.7650	4678	9.66 s	0.0197 s
CNN	0.8487	0.8632	0.8487	0.8453	0.8486	52806	9.14 s	0.0456 s
LSTM	0.7913	0.7943	0.7850	0.7868	0.7919	17542	11.20 s	0.0466 s

Table 1: Test metrics, trainable parameters, total training time and inference time on the test set.

Predictive performance

The **DNN** is clearly the best model, with an accuracy of 0.952 and a macro F1 of 0.952. The **CNN** comes second (macro F1 0.845), followed by the **LSTM** (0.787) and the **RNN** (0.756). The gap between the DNN and the other models is large, around 10 points of macro F1 with respect to the CNN.

For every model, macro and weighted F1 are very close, and precision and recall are similar. This is consistent with the test set being roughly balanced (between 420 and 537 samples per class), so no single class dominates the averages.

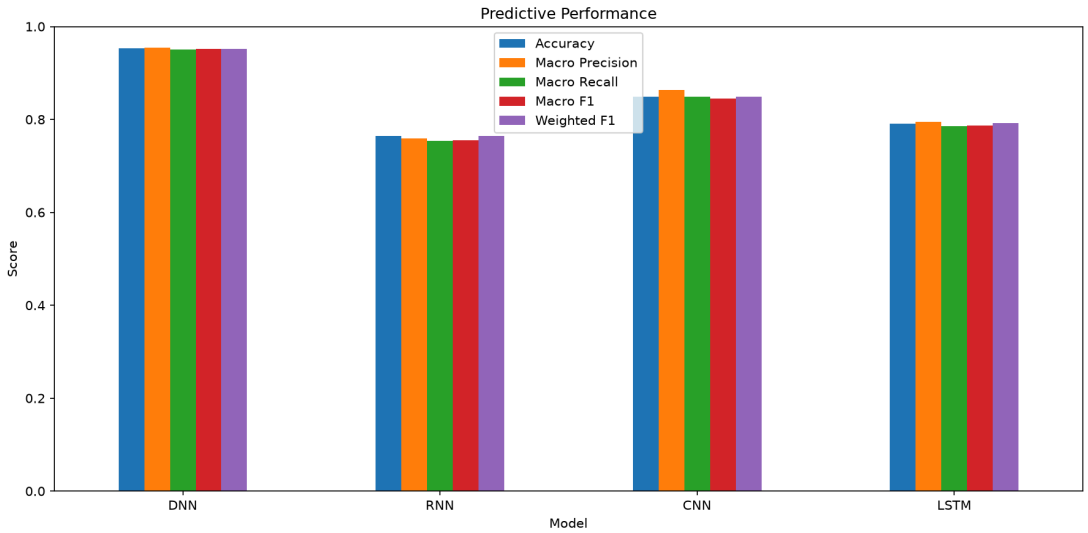


Figure 1: Predictive metrics per model.

The ranking suggests that, for these features, treating the 561 values as a single vector works better than treating them as a sequence. The features are engineered statistics (means, energies, frequency coefficients…), so their order does not carry the temporal meaning that recurrent and convolutional layers are designed to exploit.

Computational cost

The DNN has by far the most parameters (178,310, against 52,806 for the CNN, 17,542 for the LSTM and only 4,678 for the RNN), yet it is the fastest model both to train (6.6 s) and to run at inference (0.003 s for the whole test set). A fully connected layer processes the full input in a single matrix product, which is very efficient on a GPU. The CNN and LSTM are about 15 times slower at inference than the DNN.

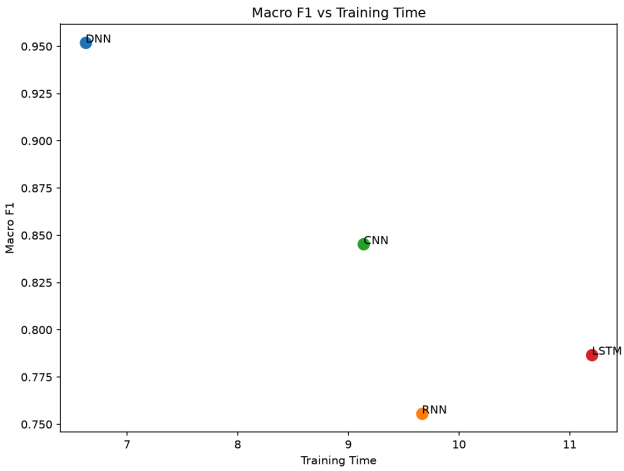


Figure 2: Macro F1 vs. training time.

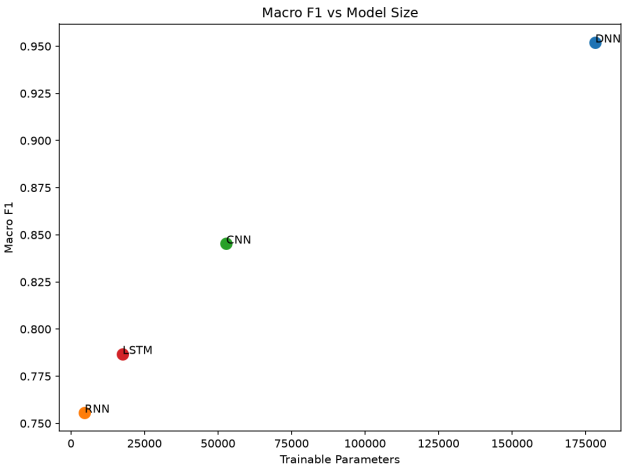


Figure 3: Macro F1 vs. number of trainable parameters.

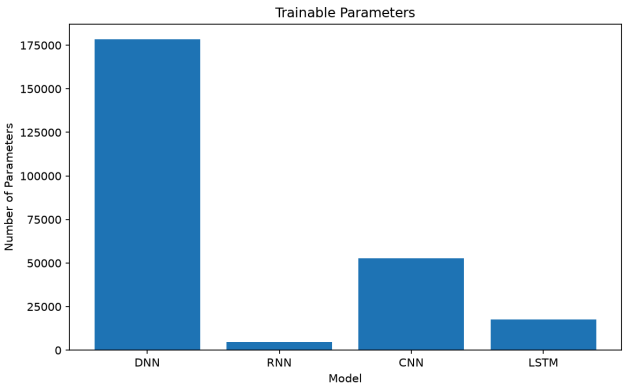


Figure 4: Trainable parameters.

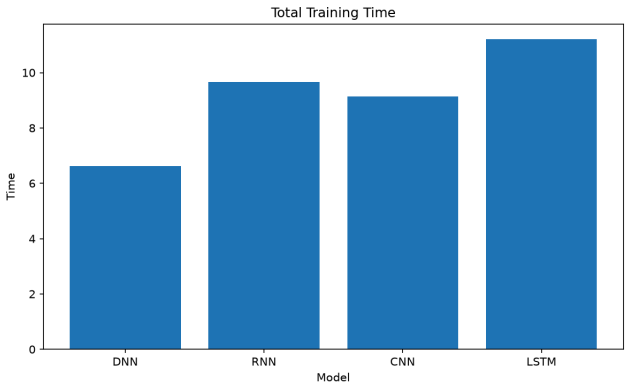


Figure 5: Total training time.

Total training time depends on both the cost per epoch and the number of epochs before early stopping. The RNN has the cheapest epochs (0.28 s) but needed 34 epochs, while the DNN and the CNN stopped after 20 and the LSTM after 27. No model reached the limit of 50 epochs.

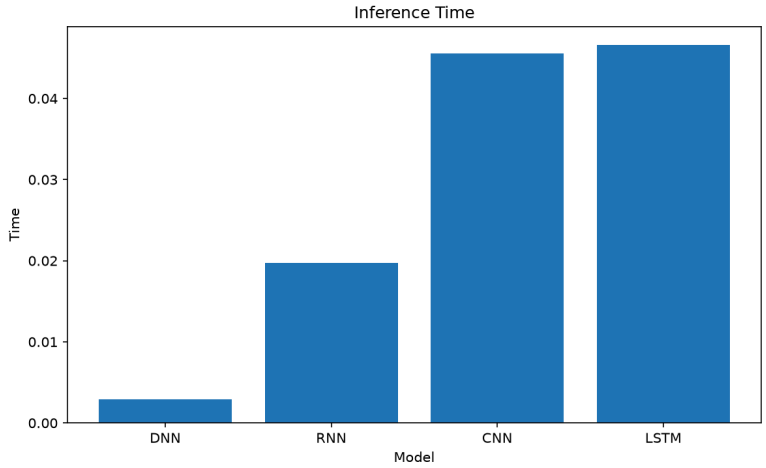


Figure 6: Inference time on the test set.

In this experiment, a larger model is not a more expensive one: the DNN offers the best accuracy at the lowest time cost, while the smaller recurrent models are both slower and less accurate.

Training curves

The DNN converges in a few epochs, and its validation loss stays low and close to the training loss. The RNN and the LSTM also learn steadily, although they converge more slowly and their validation loss is noisier.

The CNN shows the most unstable behaviour. Its training loss keeps decreasing, but the validation loss oscillates strongly, with peaks of up to about 1.7. This points to sensitivity to the learning rate or some overfitting, and it explains why its test performance is below what its training accuracy would suggest.

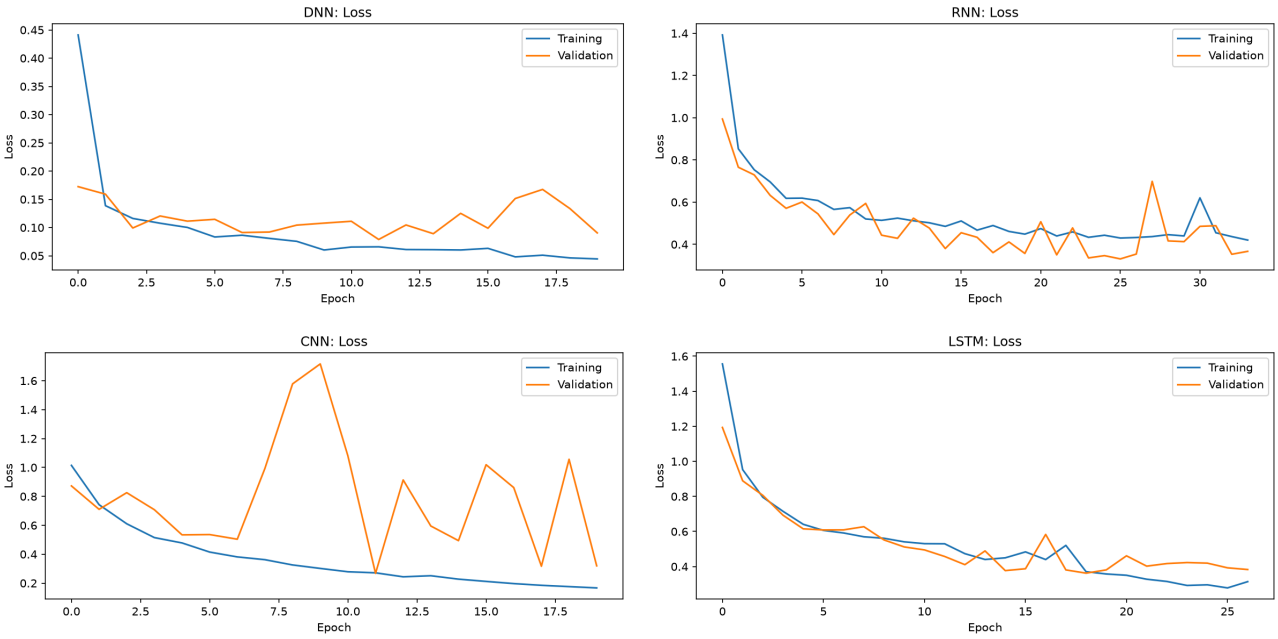


Figure 7: Training and validation loss (DNN, RNN, CNN, LSTM).

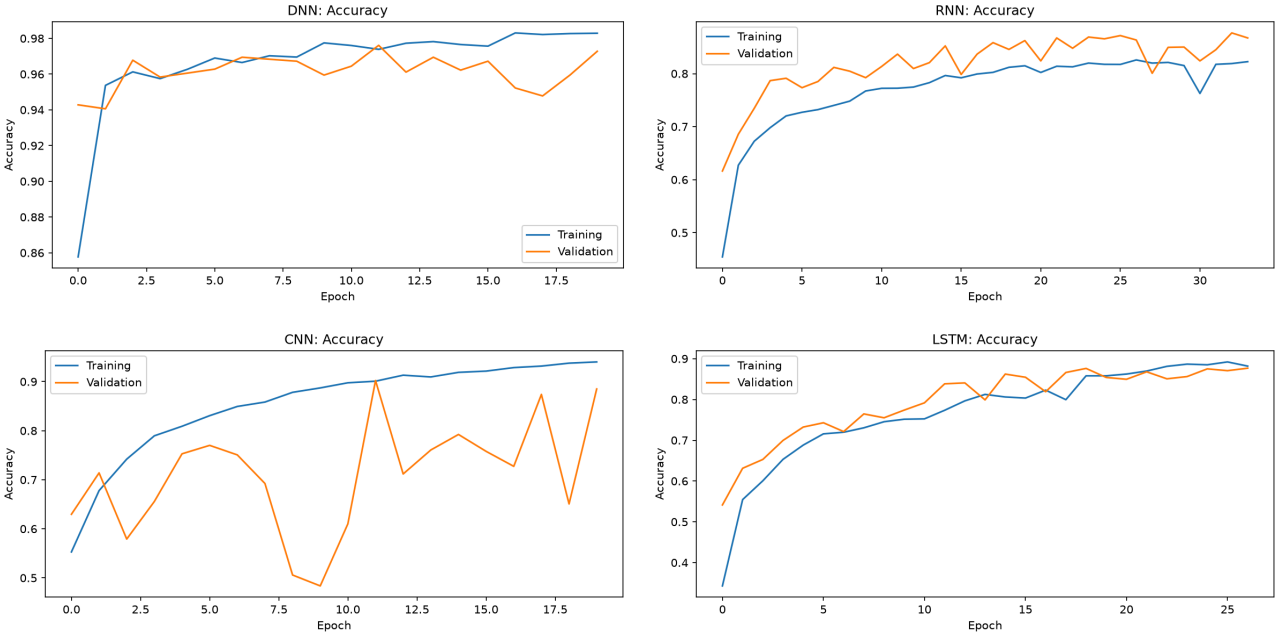


Figure 8: Training and validation accuracy (DNN, RNN, CNN, LSTM).

Confusion matrices

LAYING is recognised almost perfectly by all models (the DNN classifies all 537 samples correctly). The errors concentrate in two groups of similar activities:

- **Static activities (SITTING vs. STANDING).** The DNN still confuses 56 SITTING samples as STANDING. The LSTM confuses 114 in the same direction, and the CNN fails the other way, labelling 133 STANDING samples as SITTING.
- **Dynamic activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS).** This is where the recurrent models lose most of their accuracy: the RNN reaches an F1 of only 0.57 on WALKING_DOWNSTAIRS, and the LSTM labels 139 WALKING_UPSTAIRS samples as WALKING. The CNN also struggles with WALKING_UPSTAIRS (recall 0.66).

Errors between static and dynamic activities are much less frequent than errors within each group (the DNN makes almost none), so all models learn this coarse separation; the differences lie mainly in distinguishing activities within each group.

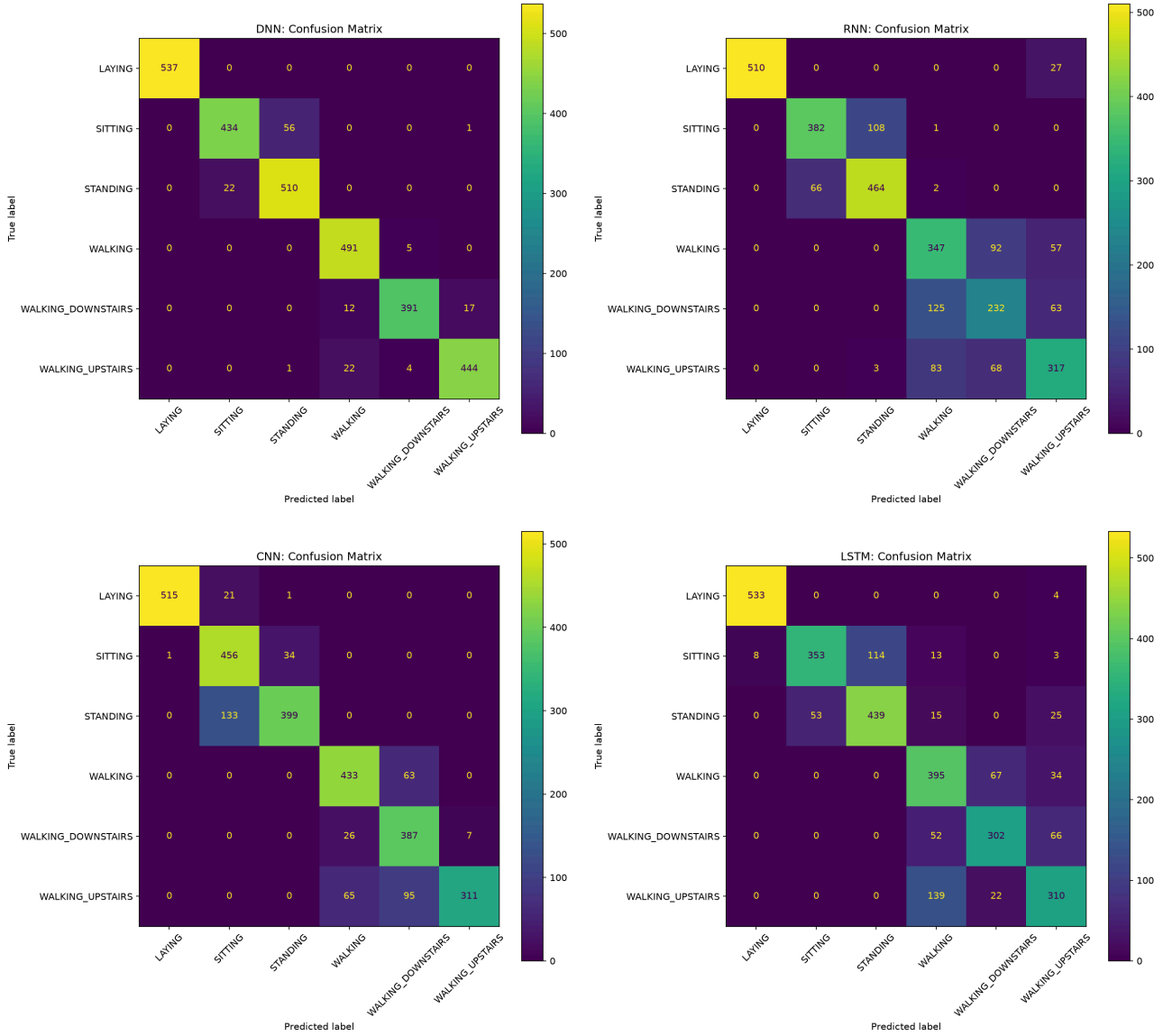


Figure 9: Test confusion matrices (DNN, RNN, CNN, LSTM).

Repeated runs with three seeds

Why. A single training run depends on random factors: the weight initialisation, the order of the mini-batches and the dropout masks. To check whether the ranking above is a property of the architectures or just of seed 42, the four models were retrained from scratch with seeds 1, 42 and 123, creating new models in every run.

Expected behaviour. Models with a smooth optimisation landscape and a strong inductive fit to the data should give similar results across seeds. Models that train on a long sequence (RNN, LSTM) or with an unstable validation loss (CNN) are expected to vary more, because small differences early in training can lead to different solutions and to early stopping at different epochs.

Model	Accuracy	Macro F1	Training Time (s)	Epochs
DNN	0.9421 ± 0.0093	0.9420 ± 0.0090	5.12 ± 0.93	13, 20, 15
RNN	0.7647 ± 0.0209	0.7559 ± 0.0205	9.20 ± 0.72	28, 34, 31
CNN	0.8298 ± 0.0205	0.8194 ± 0.0254	10.34 ± 0.87	22, 20, 23
LSTM	0.8198 ± 0.0287	0.8146 ± 0.0282	12.89 ± 0.63	31, 27, 26

Table 2: Mean ± standard deviation over seeds 1, 42 and 123 (test set). Epochs are listed per seed.

Results. The DNN is the best model with every seed (macro F1 between 0.935 and 0.952) and has the lowest standard deviation (0.009), so its advantage is clearly larger than the run-to-run noise. The RNN is the worst model in every run. The CNN and the LSTM, however, change places: with seed 42 the CNN is clearly better

(0.845 vs. 0.787), but with seeds 1 and 123 the LSTM wins, and their means are almost equal (0.819 vs. 0.815). Their standard deviations (0.025–0.028) are of the same size as the difference between them.

The seed also changes how long training lasts: the DNN stops between 13 and 20 epochs and the RNN between 28 and 34, so the training times vary accordingly. The seed-42 results match the main comparison exactly, which confirms that the pipeline is reproducible.

Conclusion. The large gaps (DNN first, RNN last) are robust, but a conclusion based on a single run would not be reliable for models whose difference is a few points, such as CNN vs. LSTM. With seed 42 alone we would have concluded that the CNN is clearly better than the LSTM, which the other seeds do not support.

Controlled experiment: LSTM hidden size

Parameter and motivation. The hidden size of the LSTM was varied over 32, 64, 128 and 256, keeping the rest of the configuration fixed (same data, optimiser, learning rate, batch size, early stopping and seed 42). It was chosen because it is the main capacity parameter of the LSTM: the whole 561-step sequence must be summarised in a single hidden state of this size. It also has a clear effect on cost, since the number of LSTM parameters grows roughly with the square of the hidden size ($4h(h+3)+6h+6$ for one input value per step).

Expected behaviour. A larger hidden state could store more information about the sequence and improve the results, especially since the LSTM is far behind the DNN. On the other hand, a larger model may overfit the 16 training subjects, and every step of the recurrence becomes more expensive, so training and inference times should grow quickly.

Hidden Size	Accuracy	Macro F1	Best Val Loss	Training Time	Epochs	Inference Time	Parameters
32	0.8127	0.8061	0.3161	11.65 s	38	0.0173 s	4678
64	0.8137	0.8074	0.3073	16.00 s	32	0.0594 s	17542
128	0.8174	0.8127	0.2850	48.08 s	27	0.3129 s	67846
256	0.8154	0.8093	0.3431	92.91 s	21	0.4518 s	266758

Table 3: LSTM with different hidden sizes (test metrics, seed 42).

Effect on performance. The hidden size has almost no effect: the macro F1 stays between 0.806 and 0.813, a range smaller than the seed-to-seed variation of the LSTM (standard deviation 0.028). The best validation loss improves slightly up to 128 and gets worse at 256, which also stops earlier (21 epochs). This suggests that the largest model starts to overfit or becomes harder to optimise, and that larger settings do not consistently improve generalisation.

Effect on cost. The cost grows much faster than the performance. From 32 to 256 the number of parameters grows about 57 times (4,678 to 266,758), the training time about 8 times (11.7 s to 92.9 s) and the inference time about 26 times. Note that the run with hidden size 64 does not reproduce the LSTM of the main comparison (macro F1 0.807 vs. 0.787) even with the same seed: there the four models are created one after another after a single call to `set_seed`, so the LSTM starts from a different initialisation. This is another example of the seed sensitivity seen above.

Conclusion. The limiting factor of the LSTM is not its capacity but the input representation: reading 561 engineered features one by one as if they were a time series does not give the recurrent model useful temporal structure. A small hidden size (32–64) is the best trade-off between performance and cost.